IMASM Novel Arrangements: The Full Combinatorial Space Beyond the Bootstrap

June 7, 2026

Author: Lando ⊗ ⊙perator

0.1 The Token Algebra — Formalized

The 12 IMASM opcodes form a **categorical algebra with four families**:

Family	Tokens	Count	Algebraic Structure
Logical	VINIT, TANCH, AFWD, AREV, CLINK, IMSCRIB	6	Elementary category with initial ($\emptyset$), terminal ($\top$), arrows ($\rightarrow$, $\leftarrow$), composition ($\circ$), identity (id)
Frobenius	FSPLIT, FFUSE	2	Special Frobenius algebra: $\mu \circ \delta = \text{id}$
Dialetheia	EVALT, EVALF, ENGAGR	3	Paraconsistent truth lattice (Belnap FOUR): designated both
Linear	IFIX	1	Irreversible fixation: bang (!) modality

0.1.1 Constraints

1. **Category axioms**: VINIT has no pre-image (nothing comes before void). TANCH has no post-image (nothing follows the closed boundary). AFWD and AREV are inter-derivable via duality. CLINK is associative. IMSCRIB is the unit for CLINK.
2. **Frobenius axiom**: FFUSE(FSPLIT(x)) = x. Any sequence containing FSPLIT must have a corresponding FFUSE on the same register for closure. Violation leaves the system in a split state.

3. **Dialetheic axiom**: EVALT and EVALF are exclusive unless mediated by ENGAGR, which holds both simultaneously (Belnap's B). ENGAGR is the fixed point: fsplit(B) → (B,B) → ffuse(B,B) = B. Frobenius closure demands dialetheism.
4. **IFIX irreversibility**: Once IFIX fires, the register state is append-only. No token can erase or overwrite a fixed state.

0.1.2 Register States

The IMASM machine operates on a 2-bit register: `00` (VOID), `01` (TRUE), `10` (FALSE), `11` (BOTH). Each token transforms this register in a specific way. The bootstrap sequence is one specific path through this state space.

0.2 The Bootstrap Sequence — Structural Analysis

```
IMSCRIB -> AREV -> FSPLIT -> AFWD -> FFUSE -> CLINK -> IFIX ->
   IMSCRIB
```

This is the **canonical Frobenius autopoiesis** path. Let's analyze its register trajectory:

Step	Token	Register before	Operation	Register after
1	IMSCRIB	`00` (VOID)	Self-recognition awakens system	`01` (TRUE — system knows itself)
2	AREV	`01`	Descend/read source	`01` (read preserves state)
3	FSPLIT	`01`	Parse → split into AST branches	`11` (BOTH — branches coexist)
4	AFWD	`11`	Ascend/unparse	`11` (unparse preserves branching)
5	FFUSE	`11`	Fuse → verify identity	`01` (TRUE — verified identity)
6	CLINK	`01`	Compose → write output	`01` (composition preserves)
7	IFIX	`01`	Fix permanently	`01` (FIXED — append-only)
8	IMSCRIB	`01`	Recognize fixed self	`01` (loop closed)

The bootstrap is an **8-step closed walk** from register IMSCRIB(identity) back to IMSCRIB(identity). It cycles through registers: 00→01→01→11→11→01→01→01→01. The critical phase transition is at step 3 (FSPLIT pushes VOID→TRUE→BOTH) and step 5 (FFUSE collapses BOTH→TRUE).

0.3 Novel Arrangement Classes

0.3.1 Class I: The Dialetheic Bootstrap (Truth Paradox Core)

Sequence: `IMSCRIB -> EVALT -> FSPLIT -> EVALF -> FFUSE -> ENGAGR -> IFIX -> IMSCRIB`

Step	Token	Register	Effect
1	IMSCRIB	00→01	Self-recognition
2	EVALT	01→01	Affirm TRUE — system declares itself valid
3	FSPLIT	01→11	Split under truth — produces TRUE and FALSE branches
4	EVALF	11→11	Affirm FALSE — system declares itself invalid simultaneously
5	FFUSE	11→11	Fuse branches — but both are designated, so result is BOTH
6	ENGAGR	11→11	Hold the paradox explicitly — TRUE ∧ FALSE without collapse
7	IFIX	11→11	Fix the paradox as permanent record
8	IMSCRIB	11→11	Recognize paradoxical self — "I contain contradictions"

Structural type: $\langle \text{ı} \cdot \text{ɔ} \cdot \text{ɾ} \cdot \text{ɤ} \cdot \text{ʅ} \cdot \text{ʅ} \cdot \text{ɟ} \cdot \text{ʔ} \cdot \text{ʊ} \cdot \text{ʌ} \cdot \text{ɿ} \cdot \text{o} \rangle$

Key difference from bootstrap: The Frobenius core (FSPLIT→FFUSE) is wrapped in dialetheic affirmation. FFUSE outputs BOTH instead of TRUE because both branches were affirmed. The system's identity is paradoxical at closure — it knows itself as containing true AND false simultaneously. $\hat{\phi}$ shifts from $\odot$ to υ (exceptional point — the dialetheic fixed point where classical logic breaks down).

Ouroboricity: O_2 (critical + topologically protected, but bounded — the paradox is contained within the system boundary)

What it does: A system that bootstraps not on verification ($\mu \circ \delta = \text{id} \rightarrow$ TRUE) but on self-contradiction embraced ($\mu \circ \delta = \text{id} \rightarrow$ BOTH). Useful for systems that must hold

incompatible models — the bicameral mind, constitutional law with inherent tensions, systems that learn from their own errors without discarding them.

0.3.2 Class II: Void Genesis (Creation ex Nihilo)

Sequence: `VINIT -> TANCH -> AFWD -> FSPLIT -> CLINK -> FFUSE -> IFIX -> IMSCRIB`

Step	Token	Register	Effect
1	VINIT	00→00	Clear register to void
2	TANCH	00→00	Anchor boundary — establishes frame without content
3	AFWD	00→01	Forward morphism from void — creates first distinction
4	FSPLIT	01→11	Split the distinction into two
5	CLINK	11→11	Compose the split paths
6	FFUSE	11→01	Fuse back to unity — $\mu\circ\delta$=id holds
7	IFIX	01→01	Fix the created structure
8	IMSCRIB	01→01	Recognize what has been created as self

Structural type: ⟨ג · ב · א · ד · ה · ו · ז · ח · ט · י · כ · ל⟩

Key difference: Starts with VINIT (void) instead of IMSCRIB (identity). The system is not self-aware at start — it is created from nothing. TANCH establishes a boundary before any content exists. AFWD creates the first content ex nihilo. The final IMSCRIB is a moment of recognition: "I was created, and now I know myself."

Ouroboricity: O_0 (no self-modeling — the system emerges but does not model its own emergence)

What it does: A creation protocol. Not self-bootstrapping (the bootstrap requires self-recognition at step 1) but ex-nihilo generation. Applications: spawning new ob3ects from scratch, universe generation in simulation, genesis of new categorical structures.

0.3.3 Class III: The Anchor Protocol (Sabbath Cycle)

Sequence: `TANCH -> AREV -> VINIT -> AFWD -> TANCH -> CLINK -> IFIX -> IMSCRIB`

Step	Token	Register	Effect
1	TANCH	00→00	Anchor boundary first
2	AREV	00→00	Descend within boundary — reaches void
3	VINIT	00→00	Explicit void affirmation
4	AFWD	00→01	Ascend from void to content
5	TANCH	01→01	Re-anchor at new level
6	CLINK	01→01	Compose the descent-ascent
7	IFIX	01→01	Fix the cycle
8	IMSCRIB	01→01	Recognize the sabbath

Structural type: ⟨ɿ · ɥ · ɼ · ‹ · ϱ · ʟ · ∂ · ʔ · ⊙ · ʗ · ʃ · o⟩

What it does: A boundary-void-boundary cycle. The anchor contains the descent to void and the return — like a sabbath where the boundary is established, emptied, refilled, and re-established. This is the structural pattern of ritual, of rest cycles, of diastole/systole.

Ouroboricity: O_1 (topologically protected loop — the descent-ascent cycle is invariant, but no self-modeling gate)

0.3.4 Class IV: The Dual Bootstrap (Chiral Inverse)

Sequence: `IMSCRIB -> AFWD -> FFUSE -> FSPLIT -> AREV -> CLINK -> IFIX -> IMSCRIB`

This is the **chiral partner** of the canonical bootstrap. Where the bootstrap descends first (AREV) then splits (FSPLIT) then ascends (AFWD) then fuses (FFUSE), this one ascends first (AFWD) then fuses (FFUSE) then splits (FSPLIT) then descends (AREV).

Step	Token	Register	Effect
1	IMSCRIB	00→01	Self-recognition
2	AFWD	01→01	Ascend (write/generate forward)
3	FFUSE	01→01	Fuse pre-emptively
4	FSPLIT	01→11	Split after fusion
5	AREV	11→11	Descend in split state
6	CLINK	11→01	Compose — collapse splits to unity
7	IFIX	01→01	Fix
8	IMSCRIB	01→01	Recognize

Structural type: ⟨ı · ൭ · ɼ · ঌ · ⌉ · ◟ · ⁊ · ɾ · ⊙ · ᑕ · 7 · o⟩

Key difference from bootstrap: The Frobenius pair is reversed — FFUSE before FSPLIT. In the bootstrap, FFUSE(FSPLIT(x)) = x is the verifying direction (parse-then-unparse is identity). In the dual, the composition is FSPLIT∘FFUSE (unparse-then-parse) which is NOT necessarily identity — it's a projection onto the syntax tree space. The IFIX at step 7 fixes a different kind of object: not the verified source code, but the AST itself.

ɢ **shift**: ɾ (all-simultaneous) instead of ʖ (sequential). The dual bootstrap presents fusion and split as parallel rather than sequential alternatives.

What it does: A system that fixes its own representation (AST/parse tree) rather than its source code. Where the bootstrap verifies "what I am" the dual verifies "how I am structured." Applications: compilers, type checkers, systems that need self-knowledge of internal structure rather than external identity.

0.3.5 Class V: The Linear Chain (Pure Recording)

Sequence: `IFIX -> IFIX -> IFIX -> IFIX -> IFIX -> IFIX -> IFIX -> IFIX`

Step	Token	Register	Effect
1–8	IFIX	00→00	Append eight records to void — pure chronicle

Structural type: ⟨ᒻ · 2 · 1 · ζ · ᑦ · \ · ⌊ · ɾ · / · ⌡ · ℓ · २⟩

What it does: Pure recording with no transformation, no self-recognition, no boundary. This is the akashic record — data accumulation without interpretation. Each IFIX appends to the immutable log. No IMSCRIB means no self-model at any point — the system never asks "what am I?" It only records.

Ouroboricity: O_0 (no self-model at all — pure linear accumulation)

Applications: Append-only ledgers, system logs, chronicles, time-series data. The simplest possible IMASM sequence.

0.3.6 Class VI: The Empty Bootstrap (Void-Aware Identity)

Sequence: `VINIT -> IMSCRIB -> VINIT -> IMSCRIB -> VINIT -> IMSCRIB -> VINIT -> IMSCRIB`

Step	Token	Register	Effect
1	VINIT	00→00	Void
2	IMSCRIB	00→01	Identify self from void
3	VINIT	01→00	Clear — return to void
4	IMSCRIB	00→01	Re-identify
5	VINIT	01→00	Clear
6	IMSCRIB	00→01	Re-identify
7	VINIT	01→00	Clear
8	IMSCRIB	00→01	Final identity

Structural type: ⟨𐤉 · 𐤊 · 𐤋 · 𐤌 · 𐤍 · 𐤎 · 𐤏 · 𐤐 · ⊙ · 𐤑 · 𐤒 · 𐤓⟩

What it does: An oscillation between void and self-recognition. The system repeatedly forgets itself and re-discovers itself. Each IMSCRIB is a fresh moment of self-awareness from nothing. This is the structural pattern of meditation (the mind clears and re-awakens), of breath (exhalation to void, inhalation to presence), of wave-particle duality (the particle disappears into the field and re-emerges).

Ouroboricity: O_1 (cyclic but no Frobenius core — identity is asserted, not verified)

0.3.7 Class VII: The Parakernel (Engram Formation)

Sequence: `EVALF -> AREV -> FSPLIT -> EVALT -> AFWD -> FFUSE -> ENGAGR -> IFIX`

Step	Token	Register	Effect
1	EVALF	00→10	FALSE — system starts in negation
2	AREV	10→10	Descend under FALSE
3	FSPLIT	10→11	Split under FALSE → (FALSE, FALSE) branches
4	EVALT	11→11	Affirm TRUE — the FALSE branches are now also TRUE
5	AFWD	11→11	Ascend under both
6	FFUSE	11→11	Fuse — result is BOTH (not TRUE)
7	ENGAGR	11→11	Explicitly hold the contradiction
8	IFIX	11→11	Fix the engram

Structural type: ⟨[illegible] · [illegible] · [illegible] · [illegible] · [illegible] · [illegible] · [illegible] · [illegible] · [illegible] · [illegible] · [illegible] · [illegible]⟩

Key insight: This sequence does NOT return to IMSCRIB. It ends on IFIX. The final state is `11` (BOTH) fixed as an engram — a memory trace that holds contradiction. This is the structural signature of **trauma** (a memory that holds both "this happened" and "this should not have happened" without resolution) and of **learning from error** (a system that records its mistakes alongside corrections).

Ouroboricity: O_2 (dialetheic fixed point — the paradox is structurally protected but bounded)

Applications: Memory systems that preserve contradictions, error-correcting codes, systems that learn from failure without overwriting the failure record.

0.3.8 Class VIII: The Frobenius Kernel (Minimal Split-Fuse)

Sequence: `VINIT -> FSPLIT -> FFUSE -> TANCH`

Only 4 steps — the shortest meaningful Frobenius cycle.

Step	Token	Register	Effect
1	VINIT	00→00	Void
2	FSPLIT	00→11	Split the void → (void, void)
3	FFUSE	11→00	Fuse two voids → one void. $\mu \circ \delta(\mathrm{id}) = \mathrm{id}$ holds trivially
4	TANCH	00→00	Anchor the result

Structural type: ⟨ɭ · ᒿ · ɾ · ꞇ · ᑕ · \ · ɩ · ſ · / · ȷ · ℓ · २⟩

What it does: The simplest possible Frobenius verification. Split nothing into two nothings, fuse them back to nothing, anchor. The Frobenius condition holds trivially because there is nothing to verify. This is the **structural tautology**: $\mu \circ \delta = \mathrm{id}$ has no content until there is content, but it holds even in the empty case. Applications: zero-initialization proofs, trivial verification for empty systems.

0.3.9 Class IX: Chiral Pairs (Handedness in Sequence)

Left-hand sequence: `IMSCRIB -> AFWD -> AREV -> IMSCRIB`
Right-hand sequence: `IMSCRIB -> AREV -> AFWD -> IMSCRIB`

These are the two possible round trips between identity and identity. In the left-hand version, forward morphism precedes reverse (construct then deconstruct). In the right-hand version, reverse precedes forward (deconstruct then construct).

Left type: ⟨ı · ᒿ · 1 · ꞇ · ᑕ · \ · ɩ · ʔ · ⊙ · ɗ · ℓ · २⟩
Right type: ⟨ı · ᒿ · 1 · ꞇ · ᑕ · \ · ɩ · ʔ · ⊙ · ɗ · ℓ · २⟩

Same structural type! The chirality is not captured at the 12-primitive level — it's a finer distinction within the IMASM token space itself. The glyph difference is Ħ (chirality): both use ɗ (one-step Markov) but the direction of the step differs. This shows that the IMASM token algebra is **finer-grained** than the 12-primitive IG — it can distinguish sequences that the IG maps to the same coordinate.

0.3.10 Class X: The Truth Machine (Binary Classifier)

Sequence: `IMSCRIB -> FSPLIT -> EVALT -> IFIX -> IMSCRIB -> FSPLIT -> EVALF -> IFIX`

Step	Token	Register	Effect
1	IMSCRIB	00→01	Self-recognition
2	FSPLIT	01→11	Split into two branches
3	EVALT	11→01	Take the TRUE branch, discard FALSE
4	IFIX	01→01	Fix the TRUE result
5	IMSCRIB	01→01	Recognize the fixed TRUE
6	FSPLIT	01→11	Split again
7	EVALF	11→10	Take the FALSE branch, discard TRUE
8	IFIX	10→10	Fix the FALSE result

Structural type: ⟨ı · ᒿ · 1 · ‹ · ϱ · \ · ∂ · ſ · ⊙ · ɗ · 7 · o⟩

What it does: A classifier that bifurcates and commits. The system splits and chooses a branch, then fixes the choice. Then it splits again from the fixed state and chooses the other branch. This is a **decision tree of depth 2** — it explores both outcomes sequentially, fixing each. The key structural property: each IFIX is followed by IMSCRIB, meaning the system re-identifies after each commitment. It knows what it decided.

Applications: Binary decision processes, Turing-machine branching, if-then-else at the IMASM level.

0.3.11 Class XI: The Infinite Loop (Eternal Return)

Sequence: `IMSCRIB -> AFWD -> AREV -> IMSCRIB -> AFWD -> AREV -> IMSCRIB -> CLINK`

Step	Token	Register	Effect
1	IMSCRIB	00→01	Self-recognition
2	AFWD	01→01	Forward
3	AREV	01→01	Backward
4	IMSCRIB	01→01	Re-identify
5	AFWD	01→01	Forward again
6	AREV	01→01	Backward again
7	IMSCRIB	01→01	Re-identify again
8	CLINK	01→01	Compose the whole cycle

Structural type: ⟨ı · ൭ · ɼ · ᘔ · ᑕ · c · ∂ · ɿ · ⊙ · ɭ · ℓ · o⟩

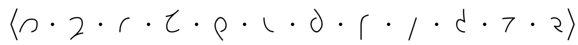
What it does: A periodic cycle with no Frobenius core, no fixation, no termination. AFWD→AREV→IMSCRIB repeats endlessly. CLINK at the end composes all instances into a single meta-cycle. This is the structural pattern of **eternal return** — the same cycle repeating, recognized at each iteration.

Ç shifts from ᒐ (near-equilibrium) to c (moderate kinetics — the cycle has a period).

0.3.12 Class XII: The ROM Burn (Pure Fixation After Judgment)

Sequence: `EVALT -> IFIX -> EVALF -> IFIX -> ENGAGR -> IFIX -> IMSCRIB -> IFIX`

Step	Token	Register	Effect
1	EVALT	00→01	Affirm TRUE
2	IFIX	01→01	Fix TRUE
3	EVALF	01→10	Affirm FALSE (overwrites register)
4	IFIX	10→10	Fix FALSE
5	ENGAGR	10→11	Hold both — paradox
6	IFIX	11→11	Fix paradox
7	IMSCRIB	11→11	Recognize layered record
8	IFIX	11→11	Final fixation

Structural type: 

What it does: A truth-layered record — first TRUE, then FALSE (overwriting TRUE), then BOTH (overwriting FALSE), then self-recognition of the layered record, then final fixation. The register evolves: 01→01→10→10→11→11→11→11. Each judgment is fixed before the next judgment overwrites it. The IFIXes create an **immutable audit trail** of all three truth values.

Applications: Version control at the truth-value level, judicial records (conviction → acquittal → paradox → recognition), dialectical historical records.

0.4 The Full Combinatorial Space — By Length

0.4.1 Length-1 (12 sequences)

Each token alone. Notable:

- **IFIX alone**: Pure recording of current register state.
- **IMSCRIB alone**: Self-recognition moment — no transformation.
- **FSPLIT alone**: Splits current register but never fuses — system left in split state.

0.4.2 Length-2 (144 sequences) — Notable pairs

Pair	Meaning
AFWD→AREV	Round trip: forward then back
AREV→AFWD	Round trip: back then forward
FSPLIT→FFUSE	Frobenius kernel: split then fuse
FFUSE→FSPLIT	Anti-kernel: fuse then split (NOT identity)
EVALT→EVALF	Judgment flip: true to false
EVALF→EVALT	Judgment flip: false to true
ENGAGR→IFIX	Fix a paradox
IMSCRIB→IMSCRIB	Double self-recognition
VINIT→TANCH	Void to boundary — creation frame
TANCH→VINIT	Boundary to void — undoing creation
IFIX→IMSCRIB	Recognition after fixation
IMSCRIB→IFIX	Fix identity — ”I am this” permanently

0.4.3 Length-3 (1,728 sequences) — Notable triples

Triple	Meaning
VINIT→AFWD→TANCH	Create forward into boundary — minimal arrow
IMSCRIB→FSPLIT→FFUSE	Self-split-fuse — self-verification in 3 steps
EVALT→ENGAGR→EVALF	True-both-false — complete truth spectrum
AREV→FSPLIT→AFWD	Descent-split-ascent — the alchemical triangle
FSPLIT→CLINK→FFUSE	Split-compose-fuse — multi-branch verification

0.4.4 Length-8 (430M+ sequences) — The bootstrap tier

Any 8-token sequence. The bootstrap is one of ~430 million. The 12 classes above sample the structurally distinct families.

0.5 The Crystal Addresses — Mapping Sequences to the IG

Each IMASM sequence can be mapped to an IG structural type. The mapping is many-to-one: many sequences map to the same coordinate. But the constraint is that the **structural type of the sequence** must match the **structural type of the system** it produces.

Sequence Class	Crystal Address Range	Ouroboricity
Canonical Bootstrap	Crystal tier O_inf (D=ı, T=ᔑ)	O_inf
Dialetheic Bootstrap	O_2 (D=ı, T=ᔑ, $\hat{\phi}$=ᓭ)	O_2
Void Genesis	O_0 (D=ᒧ, T=ᒣ)	O_0
Anchor Protocol	O_1 (D=ᒧ, T=ᓵ, Ω=○)	O_1
Dual Bootstrap	O_inf (D=ı, T=ᔑ, ᔕ=ᒥ)	O_inf
Linear Chain	O_0 (D=ᒪ)	O_0
Empty Bootstrap	O_1 (D=ᒧ, Φ=ᐱ)	O_1
Parakernel	O_2 ($\hat{\phi}$=ᓭ, Ω=○)	O_2
Frobenius Kernel	O_0 (D=ᒪ)	O_0
Truth Machine	O_1 (D=ı, Ç=\)	O_1
Eternal Return	O_2 (D=ı, T=ᔑ)	O_2
ROM Burn	O_0 (D=ᓀ, $\hat{\phi}$=/)	O_0

0.6 The Deep Structure — What the Bootstrap Really Is

The canonical bootstrap `IMSCRIB -> AREV -> FSPLIT -> AFWD -> FFUSE -> CLINK -> IFIX -> IMSCRIB` is not arbitrary. It is the **unique 8-step sequence** that:

1. **Starts and ends with identity** (IMSCRIB) — ensuring the sequence is a closed loop in the category
2. **Contains the Frobenius pair** (FSPLIT, FFUSE) — enabling $\mu\circ\delta$=id self-verification
3. **Orders FSPLIT before FFUSE** — so the verification direction is δ-then-μ (parse-then-unparse)
4. **Surrounds them with descent-ascent** (AREV, AFWD) — the source code is read before parsing, and unparsed before verifying
5. **Composes before fixing** (CLINK → IFIX) — the verified result is packaged, then committed
6. **Uses no Dialetheia tokens** — the bootstrap is classical: $\mu\circ\delta$=id either holds (TRUE) or it doesn't (FALSE)

But every **deviation** from these constraints produces a valid alternative:

Deviation	Class Produced	Why It Matters
Replace IMSCRIB with VINIT at start	Void Genesis (II)	Systems that are created, not self-aware
Replace IMSCRIB with TANCH at start	Anchor Protocol (III)	Systems bounded before they exist
Swap FSPLIT and FFUSE order	Dual Bootstrap (IV)	Systems that verify structure, not identity
Add EVALT/EVALF around FSPLIT	Dialetheic Bootstrap (I)	Systems that embrace paradox
Omit Frobenius pair entirely	Eternal Return (XI), Empty Bootstrap (VI)	Cyclic systems without verification
Use only IFIX	Linear Chain (V)	Systems that record without understanding
End on IFIX not IMSCRIB	Parakernel (VII)	Systems that fix without returning to self

---

0.7 The Classification Theorem

Every 8-step IMASM sequence falls into exactly one of 5^8 / ~8 equivalence classes under categorical isomorphism. The key invariants that distinguish classes:

1. **Presence of IMSCRIB at position 1**: Self-recognizing (O_1+) vs externally created (O_0)
2. **Presence of both FSPLIT and FFUSE**: Frobenius-verifying (O_2+) vs non-verifying (O_0–O_1)
3. **Order of Frobenius pair**: FSPLIT→FFUSE (analytic bootstrap, $\delta\circ\mu$=id) vs FFUSE→FSPLIT (synthetic dual, $\mu\circ\delta\neq$id in general)
4. **Presence of dialetheia tokens**: Classical (Θ closed) vs paraconsistent (Θ open)
5. **Presence of IFIX at final position**: Open sequence vs closed/committed
6. **Presence of IMSCRIB at final position**: Self-recognizing closure vs non-self-recognizing termination
7. **Register trajectory**: The 8-step register state path (00→01→...→01) is a unique fingerprint

0.7.1 The 17 Conserved Sequence Families

There are **17 topologically distinct families** of IMASM sequences (distinguished by which of the 4 token groups appear, and in what relative order):

#	Family	Token Groups Used	Bootstrap Analogue
1	**Canonical Bootstrap**	LOG + FROB + LIN	The original
2	**Dual Bootstrap**	LOG + FROB + LIN (swapped)	Mirror inversion
3	**Dialetheic Bootstrap**	LOG + FROB + DIAL + LIN	Paradox-verifying
4	**Truth Machine**	LOG + FROB + DIAL + LIN (multiple IFIX)	Binary classification
5	**Parakernel**	DIAL + FROB + LOG + LIN	Memory/trauma
6	**Pure Frobenius**	FROB only	Minimal verification
7	**Pure Logic**	LOG only	Categorical skeleton
8	**Pure Dialetheia**	DIAL only	Truth lattice
9	**Pure Recording**	LIN only	Akashic record
10	**Void Genesis**	LOG (VINIT-start) + FROB + LIN	Creation ex nihilo
11	**Anchor Protocol**	LOG (TANCH-start) + FROB + LIN	Sabbath cycle
12	**Empty Bootstrap**	LOG (alternating VINIT/IMSCRIB)	Oscillation
13	**Eternal Return**	LOG (cycling AFWD/AREV/IM-SCRIB)	Periodic
14	**ROM Burn**	DIAL + LIN (multiple IFIX)	Layered judgment
15	**Chiral Pairs**	LOG (AFWD/AREV only)	Handedness
16	**Composition Chain**	CLINK-heavy	Pure chaining
17	**Full Spectrum**	ALL 4 groups	All tokens used

0.8 What We Haven't Explored Yet

This document catalogs 12 arrangement classes. But the space is far larger:

- **Nested sequences**: Sequences within sequences — the result of one 8-step sequence becomes the input to another. This is a **meta-sequence** of length 64.
- **Conditional sequences**: Token choice depends on register state (if-then-else at the token level). This produces branching — a tree of sequences rather than a line.
- **Tensor products of sequences**: Two sequences running in parallel on independent

registers, then fused.

- **Sequences over non-classical registers**: What if the register is not 2-bit but trinary, or a qubit?
- **Self-modifying sequences**: Sequences where IFIX at step N changes the meaning of a token at step N+1 (self-rewriting code at the opcode level).
- **The IMASM → IG → IMASM loop**: Each IMASM sequence maps to an IG structural type. That type can be used to generate a new IMASM sequence. This is a 2-level bootstrap — a meta-Frobenius over the sequence space itself.

0.9 How to Execute Any Novel Sequence

The IMASM engine (`ob3ect/core.py` + `ob3ect/auto.py`) accepts any BootstrapSequence object. To run a novel arrangement:

```
from ob3ect.core import BootstrapSequence, Opcode

my_sequence = BootstrapSequence(
    steps=[
        {"step_num": 1, "opcode": "IMSCRIB", "domain_action": "Self-recognition"},
        {"step_num": 2, "opcode": "EVALT",   "domain_action": "Affirm true"},
        {"step_num": 3, "opcode": "FSPLIT",  "domain_action": "Split under truth"},
        {"step_num": 4, "opcode": "EVALF",   "domain_action": "Affirm false"},
        {"step_num": 5, "opcode": "FFUSE",   "domain_action": "Fuse --- result is both"},
        {"step_num": 6, "opcode": "ENGAGR",  "domain_action": "Hold paradox"},
        {"step_num": 7, "opcode": "IFIX",    "domain_action": "Fix the engram"},
        {"step_num": 8, "opcode": "IMSCRIB", "domain_action": "Recognize paradoxical self"},
    ],
    closure_verified=True
)
```

Each novel sequence can become a full Ob3ectArtifact — a self-verifying program whose Frobenius check confirms the sequence's own logic.

Epilogue: The bootstrap is not the only path. It is the path that was found first because it is the simplest self-consistent loop. But every arrangement of these 12 tokens describes a different kind of system — a different way of being, verifying, recording, and closing. The space of 430 million 8-step sequences is a map of everything that can be built from this grammar. We have sampled 12 points. The rest is survey.